

Java Section 2

TYPES AND CONTROL STATEMENTS

You will learn about variable types and how to control the flow of execution within methods by selecting statements and repeating statements.

Types and Control Statements

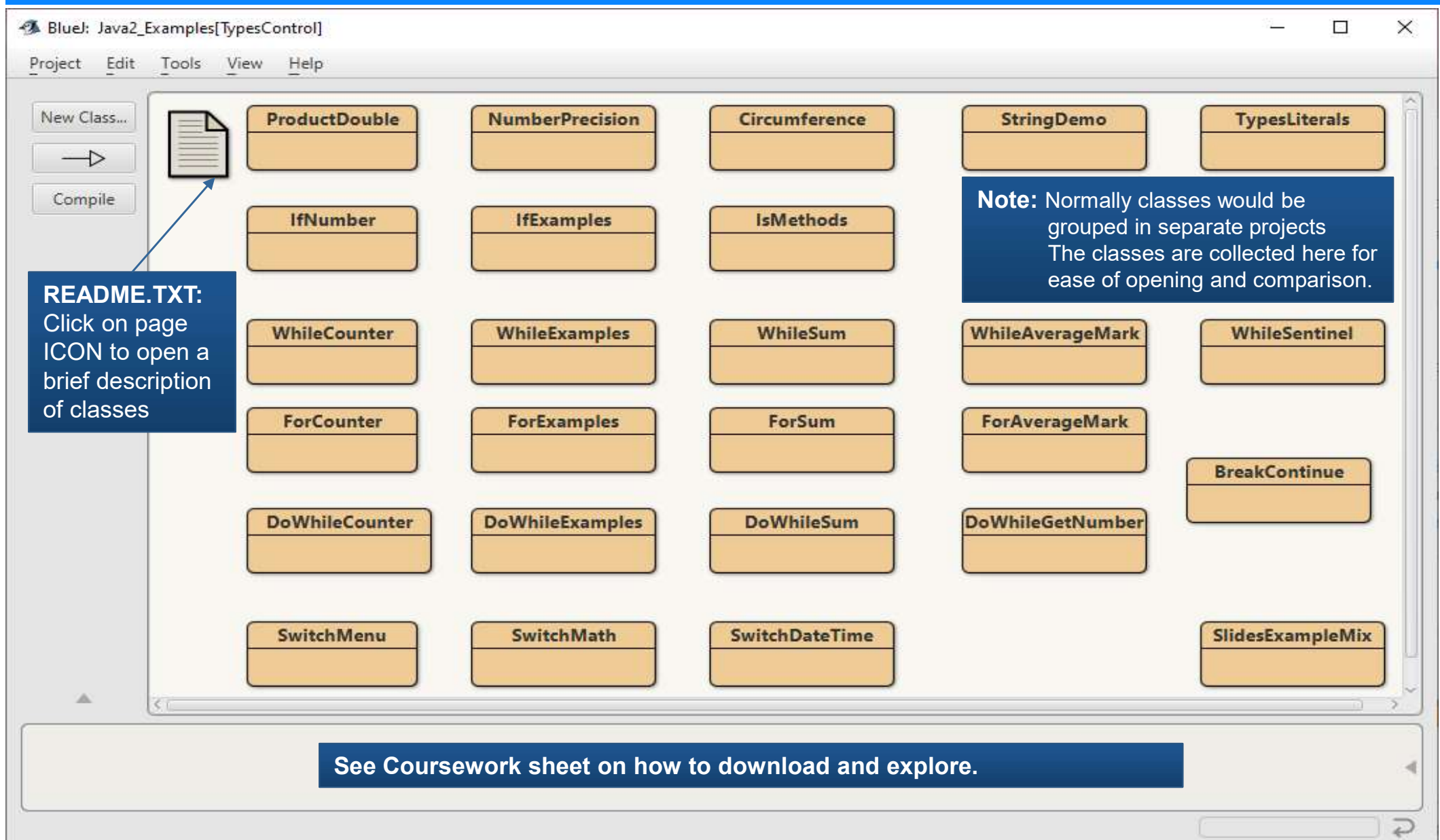
- **Java variable types: Primitive and Reference**
- **Real number primitive types**
- **Using algorithms to solve problems**
- **More on pseudocode**
- **Introduce UML activity diagrams.**
- **if, if ... else** selection statements
- **while** repetition statement
- **Counter and sentinel controlled repetition**
- **Operators:** Increment, Decrement, Compound Assignment
 Conditional, Cast, Logical (Conditional/Boolean)
- **do while, for** repetition statements
- **switch** selection statement

Lectures will emphasize the most important!

It is strongly recommended that you also read the recommending readings and complete the associated exercises. JN

BlueJ: Java2_Examples[TypesControl]

Examples discussed in this section: Go Explore!



Java Variable Types

Primitive Types

- Java is a **strongly typed language**.
 - Recall that variables must be declared before use.
 - Java requires all variables to specify their **type** when first declared.
 - Each variable must have a **type**: either a **primitive type** or a **reference type**
- Java has eight **primitive types** portable across all platforms
byte, short, int, long, float, double, char, boolean
- **Local variables** are declared and created within methods
 - They are never initialized.
 - **Initialization**: (the value a variable has when first created)

Part of Java's
WORA

Write Once,
Run Anywhere.

Examples of Primitive type local variables:

int number1;

in the **main** method of the class **FindMax**

int maximum;

in both the **main** and **max** methods of class **FindMax**
two separate distinct local variables

Java Variable Types

Reference Types

- All classes are **reference types** (*aka non-primitive types*)
 - Objects are accessed through **variables of reference types** (*aka references*)
- **Reference-type** variables are
 - always **initialized** to the value **null**.
 - used to **invoke** the methods of an object.

Examples of a reference type local variable:

```
Scanner keyboard = new Scanner(System.in);
```

in class FindMax

And invoking a method

```
int number2 = keyboard.nextInt( );
```

The String type is a reference type:

```
String country = "Ireland";
```

Java Variable Types

Real number/floating point primitive types

- Recall: Integers are normally of type **int**
- Floating-point numbers
 - Used to store **real** numbers 2.5, -8.0, 3.1415926, 4.5E3 (4.5×10^3)
 - Two primitive types: **float** and **double**
 - float** Used for general real number storage
 - double** Stores numbers with greater magnitude and precision than **float**
- Precision and Memory Requirements
 - Floating point numbers may not be stored precisely
 - **float** represents **single-precision floating-point numbers**
 - Provide up to seven significant digits.
 - A floating-point literal is suffixed with the letter **F** or **f**; 4.2**f**, 3.1415**F**
 - **double** represents **double-precision floating-point numbers**
 - Provide 15 significant digits, ~ double the precision of **float** variables.
 - Require twice as much memory to store as **float**

Java Primitive Types

Type size in Bytes and Ranges

	Type	# Bytes	Range
numeric integral	byte	1	-128..127 $-2^7 \dots 2^7-1$
	short	2	-32768..32767 $-2^{15} \dots 2^{15}-1$
	int	4	-2,147,483,648.. 2,147,483,647 $-2^{31} \dots 2^{31}-1$
	long	8	- 9,223,372,036,854,775,808 .. 9,223,372,036,854,775,807 $-2^{63} \dots 2^{63}-1$
numeric floating point	float	4	$\pm 1.4\text{e-}45 \dots 3.4\text{e+}38$ (7 digits precision)
	double	8	$\pm 4.9\text{e-}324 \dots 1.8\text{e+}308$ (15 digits precision)
Unicode	char	2	0 .. 65535 or '\u0000' to '\uffff'
	boolean		true, false

Floating point Examples

Demonstrate **double** and **printf** format specifiers

ProductDouble

ProductDouble.java

```
import java.util.Scanner;
public class ProductDouble
{
    public static void main(String[] args)
    {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter 1st number: ");
        double number1 = input.nextDouble();

        System.out.print("Enter 2nd number: ");
        double number2 = input.nextDouble();

        double product = number1 * number2;
        System.out.printf( "The product of <%.2f> and <%8.2f> is <%f>\n",
            number1, number2, product );
    }
}
```

System.out.printf("%f", ...

%f is format specifier for float/double

Options: %<flags><width><.precision>**f**

%f Display using default precision of 6 places after the decimal point

%.2f Print with 2 places after decimal point

%8.2f Use a minimum width of 8 characters, with 2 places after the decimal point, 1 for the point, and 5 before it.

<flags> include **+-0#(,**

e.g. **-** left justify, **0** to pad with zeros.

If the width too large, Java will pad with spaces.

If too small, it will extend to display the number.

[Java Tutorial: Formatting Numeric Print Output](#)

[JavadevJournal: Formatting with printf\(\) in Java](#)

Enter 1st number: 4.2

Enter 2nd number: 2.5

The product of <4.20> and < 2.50> is <10.500000>

Floating point Examples

Request a radius and calculate the circumference of a circle.

Circumference

Circumference.java

```
import java.util.Scanner;
public class Circumference
{
    public static void main(String []args)
    {
        Scanner keyboard = new Scanner( System.in );

        System.out.println("Calculate the circumference of a circle");

        System.out.print( "Enter the radius: " );
        double radius = keyboard.nextDouble( );

        double circumference = 2 * Math.PI * radius;

        System.out.println("Circumference is " + circumference + "\n");
    }
}
```

```
Calculate the circumference of a circle
Enter the radius: 10
Circumference is 62.83185307179586
```

Floating point Exercise

Methods to calculate the circumference and area of a circle

1. Write a method **calcCircumference** that takes a real number parameter **radius** and calculates and returns the circumference. Complete the **javadoc** comment.
2. Test the **calcCircumference** method by calling it in the main method with a selected value for **radius** and displaying the result.
3. See the solution in **Circumference.java**
4. Write another method called **calcArea** that accepts a **radius** and calculates and returns the area of the corresponding circle.
5. Test the **calcArea** method in the main method.
6. Compare with the **calcCircumference**

Floating point Exercise Solution

Method to calculate the circumference of a circle

The method `calcCircumference` takes a real number parameter `radius`, calculates the circumference $2\pi r$ and returns this value.

Example method calls in the main method: `System.out.println( calcCircumference(10) );`
`double circum = calcCircumference(10);`

Circumference.java

The method **calcCircumference** takes a real number parameter **radius**, calculates the circumference $2\pi r$ and returns this value.

```
/**
 * Calculate the circumference of a circle
 * @param r    the radius
 * @return     the circumference
 */
public static double calcCircumference(double r)
{
    return 2 * Math.PI * r;
}
```

Example method calls
in the main method:

```
System.out.println( calcCircumference(10) );
double circum = calcCircumference(10);
```


Floating point precision

Demonstrate floating point precision

NumberPrecision

NumberPrecision.java

- Floating point numbers may not always be represented accurately.
- In floating point calculations there are often **rounding errors**.
 - You would expect that $(n/d) * (d/n) = 1$ but in floating point, for both **float** and **double** this is not always the case.

Note: You may not understand some of the code, It will become clear later.

- Class **NumberPrecision** has 5 methods to execute:

```
accuracyFloatDoubleConstants( );           // float/double 0.1
accuracyFloatDoubleExpression( );          // float/double 1.0/3.0
findRoundingErrorsFloat(12);               // explore float accuracy
findRoundingErrorsDouble(12);              // explore double accuracy
checkForRoundingError( );                  // try various float values
```

- For **float** the expression **(11.0f/7.0f) * (7.0f/11.0f)** is not **1.0** but:

```
Fraction entered 11.0f/ 7.0f is decimal    1.571428537368774400
(11.0f/ 7.0f) x ( 7.0f/11.0f) is decimal  0.999999940395355200
The rounding error is                      0.000000059604644775
```


Floating point Examples

NumberPrecision

Demonstrate difference between float and double precision.

NumberPrecision.java

```
/**
 * Demonstrate difference of representation in float and double for 0.1
 */
public static void accuracyFloatDoubleConstants()
{
    double dPointOne = 0.1;        // Represent 0.1 as a double
    float fPointOne  = 0.1f;       // Represent 0.1 as a float

    System.out.printf( "0.1  as a double is %.18f\n", dPointOne);
    System.out.printf( "0.1f as a float  is %.18f\n", fPointOne);
    if (dPointOne != fPointOne)    // This will be true
        System.out.printf( "  The difference is %.18f\n", dPointOne - fPointOne);
}
```

- Floating point constants are stored in **double** precision e.g. **0.1**
- To define a **float** constant add **f** suffix to value e.g. **0.1f**
- **%.18f** Format specifier of 18 places after the decimal point
- What value is **d**? if **double d = 0.1f**;

```
0.1  double is 0.100000000000000000
0.1f float  is 0.100000001490116120
Difference is -0.000000001490116114
```

Floating point Examples

Demonstrate accuracy issues in expressions.

NumberPrecision

NumberPrecision.java

```
/**
 * Demonstrate difference of accuracy in float and double for 1 divide by 3
 */
public static void accuracyFloatDoubleExpression()
{
    double dThird = 1.0/3.0;           // Represent the fraction 1/3 in double
    float fThird  = 1.0f/3.0f;         // Represent the fraction 1/3 in float

    System.out.printf( "double 1/3 is %.18f\n", dThird);
    System.out.printf( "float  1/3 is %.18f\n", fThird);
}
```

- 1.0/3.0 will result in a **double** result, since both operands are **double**
- 1.0f/3.0f will result in a **float** result, since both operands are **float**
- Note in this example, there are 16 digits of precision for **double** and only 7 digits for **float**

```
double 1/3 is 0.33333333333333333300
float  1/3 is 0.3333333343267440800
```

char and boolean

Primitive types

■ char

- A data type to contain a single character
- Uses the 16-bit Unicode character set (65536 characters)
- Character literals are enclosed in single quotation marks ' ' e.g. 'A', '4'
- Can specify the hexadecimal value using '\u0000' to '\uffff'

```
char answer = 'Y';
```

```
char letter = '\u0041'; // Unicode for 'A', same as 0x41, 65
```

■ boolean

- Data type has only two possible values: **true** and **false**
- Used for tracking conditions which only can be true/false.
- Frequently prefix **is** applied to **boolean** variable names.

```
boolean isEmpty = false;
```

```
boolean isComplete;
```

```
isComplete = true;
```

Java Literals

are values specified as **constants** in the code.

	Type	Distinguisher	Examples
numeric integral	byte short int	Prefix: None, 0, 0x, 0X, 0b, 0B None Decimal <i>base 10</i> 0x 0X Hexadecimal <i>base 16</i> 0 Octal <i>base 8</i> 0b 0B Binary <i>base 2</i>	Considered <i>int</i> by default 31 0x1F 0X1F 037 0b11111 0B11111
	long	Postfix: l or L required if value is above the <i>int</i> limits	31l 31L 0x1FL <i>Optional</i> 4000000000L <i>Required</i>
numeric floating point	float	Postfix: f or F	4.2f 4.2F
	double	Postfix: None, d, or D	4.2 4.2d 4.2D
Unicode	char	Enclose Single Quotation or Integer	'A' 65 '\u0041'
	boolean	None, literals are true and false	true false
Reference	String	Enclose in Double Quotation	"A String literal"

Java Primitive Types and Literals

Examples

TypesLiterals

TypesLiterals.java

```
// Each numeric type has a maximum and minimum value, stored in the
// corresponding literal in TypeClass.MAX_VALUE and TypeClass.MIN_VALUE
// e.g. the max can be found by:
byte maxByteValue = Byte.MAX_VALUE;           // = 127; 0177; 0x7F, 0b1111111
short maxShortValue = Short.MAX_VALUE;         // = 32767;
int maxIntValue = Integer.MAX_VALUE;           // = 2147483647;
long maxLongValue = Long.MAX_VALUE;            // = 9223372036854775807L;

float maxFloatValue = Float.MAX_VALUE;         // = 3.40282346638528860e+38f;
double maxDoubleValue = Double.MAX_VALUE;      // = 1.79769313486231570e+308;

char char1 = 'A';                             // Character literal for A
char char2 = 65;                              // Character integer for A
char char3 = '\u0041';                        // Unicode literal for A

boolean isResultCorrect = (3*4==12);           // Will assign true to variable
boolean isComplete = false;

// Define literal format string for integer and real numbers
String formatIntegers = "%8s: %d\n";
String formatReals = "%8s: %.18g\n";
```

Exercise: Review the additional code given in the source code file to display the above values. Add code to also display the **minimum** values of all the numeric types.

String literals and Instantiating a String Object

- The Java **String** class represents character strings e.g. **"Hello"**
- A **String** literal is a sequence of characters enclosed in " "
 - Examples: **"Hello"**, **"Contact us!"**, **"V94 T9PX"**
 - The sequence of characters for **"Hello"** is 'H' 'e', 'l', 'l', and 'o'.
 - All String literals are implemented as instances of the **String** class.
 - A new String object is automatically created for each String literal.
- We can instantiate a String object using the keyword **new** followed by the name **String** and parentheses, with an optional argument.

String greeting = **new** String("Hello");

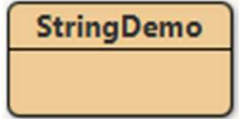
Creates a String object named **greeting** which has the attribute **"Hello"**

- Creating String objects is very common, so Java has a shorthand:

String greeting = "Hello";

Instantiating an Object of type String

Demonstrate explicit and implicit String instantiation.



StringDemo.java

```
public class StringDemo
{
    public static void main(String[] args)
    {
        String greeting = new String("Hello");
        System.out.println( greeting );

        String greetings = new String("Hello, Hola, Ciao, nǎ hǎo");
        System.out.println( greetings );

        // Since String objects are very common, a shorthand exists
        String city = "Limerick";    // we can omit the new String(...);
        System.out.println( city );
    }
}
```


Introduction to Software Design

- Most problems can be solved by identifying and executing a set of actions in a particular sequence.
- Solving problems by programming requires:
 1. A clear **understanding** of the problem.
 2. A **knowledge of the programming language** and its building blocks.
 3. A **design and plan** of how to solve the problem.
 4. The **program development**, (code, test and debug the program).
- Each **programming language** has a different syntax and different building blocks available e.g. existing classes and methods.
- The **design** can include requirements, **UML diagrams**, freehand diagrams, **flow charts**, **pseudocode**, etc
- Often the most difficult part of solving problems on a computer is identifying the requirements and developing the design(s).
- Developing the program from the design is normally straightforward once the language is mastered.

Algorithms

- An **algorithm** is a specified procedure to solve a problem
 - a sequence of definite and effective instructions or tasks, which terminates with the production of correct output from the given input.
 - Order is normally very important.
- Algorithms are common in mathematics and computer science.
 - Arithmetic algorithms for finding Greatest Common Divisor, prime numbers etc
 - Can use to specify calculations, data processing, reasoning,
- A **computer program** is essentially an elaborate **algorithm** that goes through a series of steps to arrive at a specific outcome [Intel].
- Various forms used to allow programmers to outline, express, and develop algorithms or solutions to problems:
 - **Natural language** often verbose, and difficult to write without ambiguity
 - **Pseudocode** a plain language often using structures of a programming language
 - **UML activity diagram** models both computational and organizational processes
 - **Flow chart** a type of diagram that represents a workflow or process. uses rectangles, ovals, diamonds etc and connecting arrows to define the flow and the sequence

Pseudocode and Flow Charts

Prompt the user to enter two numbers and display the maximum.

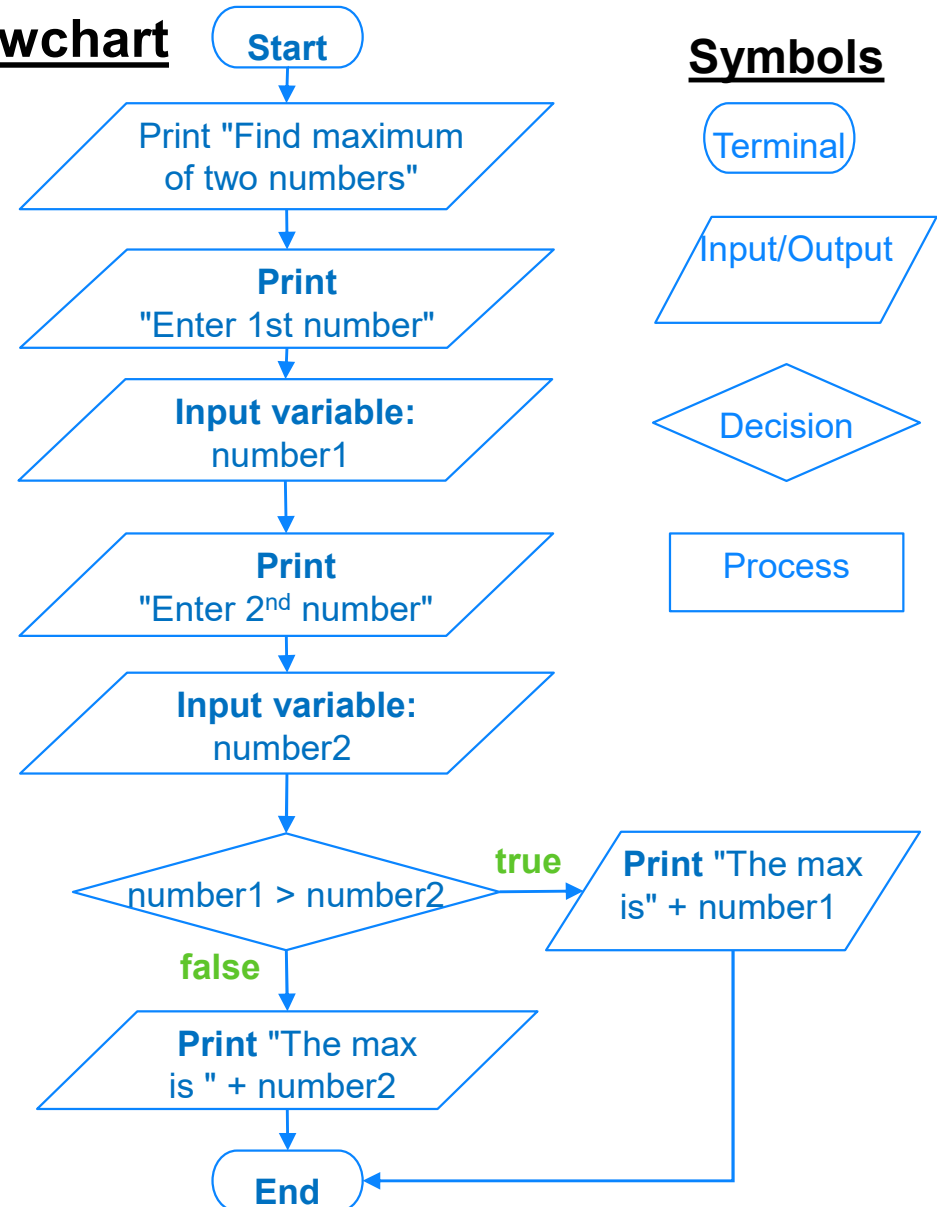
Pseudocode

1. **Print** "Find maximum of two numbers"
2. **Print** "Enter 1st number: "
3. **Input** number and store in variable number1
4. **Print** "Enter 2nd number: "
5. **Input** number and store in variable number2
6. **If** number1 >= number2
7. **Then** Print "The max is " number1
8. **Else** Print "The max is " number2

More information:

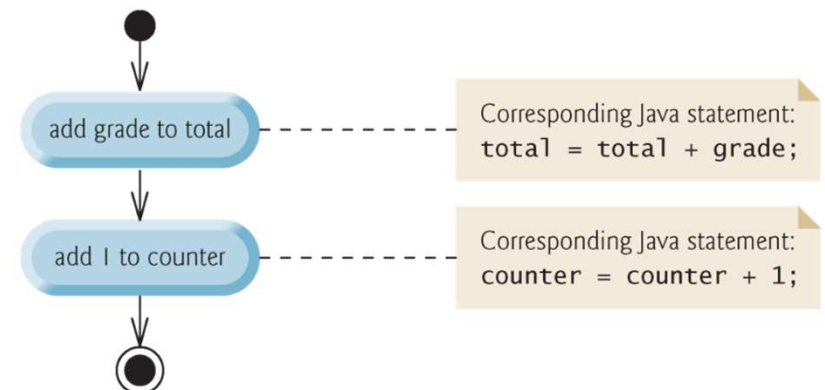
- [What is a flowchart? tutorial Lucidchart](#)
- [Flowchart \(Wikipedia\)](#)
- [Pseudocode \(Wikipedia\)](#)

Flowchart



UML Activity Diagrams

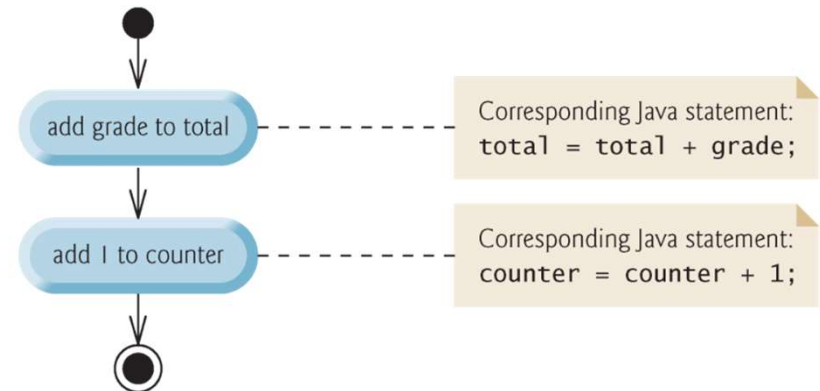
- Model the workflow (also called the activity) of a portion of software.
- Help develop and represent **algorithms**, or a portion of an **algorithm**.
- Show how the control flow operates
- Composed of symbols
 - **action-state symbols** (rectangles with outward arcs on left and right sides)
 - **diamonds**
 - **small circles**
- Symbols are connected by transition arrows, which represent the flow of the activity—the order in which the actions should occur.
- Sequence structure in UML
 - Uses connected **action-state symbols**
 - Anywhere a single action may be placed, we may place several actions in sequence.
 - This activity diagram illustrates a typical sequence structure in which two calculations are performed in order.



UML Activity Diagrams

www.uml.org

- The two **action states**
 - Represent the actions to perform.
 - Each contains an action expression that specifies a particular action to perform.
- Arrows represent **transitions**
 - order in which the actions represented by the action states occur.
- **Solid circle** represents the initial state
 - the beginning of the workflow before the program performs the modelled actions.
 - Normally at the top
- **Bullseye** (***Solid circle surrounded by a hollow circle***) represents the final state
 - Denotes the end of the workflow after the program performs its actions.
 - Normally at the bottom
- UML *notes* - Like comments in Java
 - Rectangles with the upper-right corners folded over.
 - Dotted line connects each note with the element it describes.



Basic Concepts

Variables

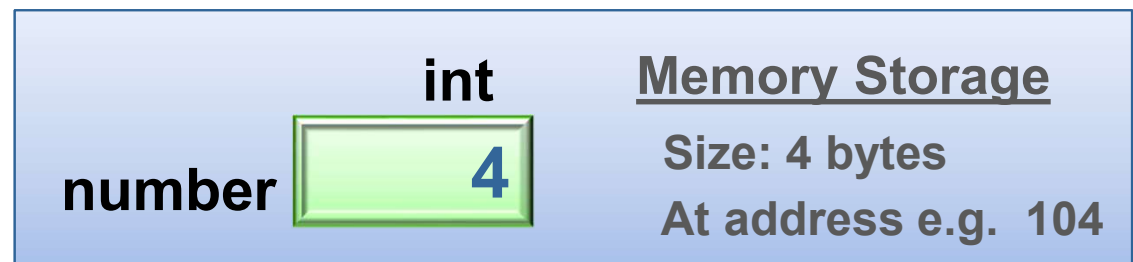
- A **variable** is used to store information. The **variable name**:
 - is a symbolic name to represent the storage location with a quantity of information.
 - refers to the **variable storage** or the **value** contained (depends on context).

Consider the variable **number** :

```
int number;
```

```
number = 4;
```

```
System.out.println( number );
```



- In Java we must declare a **variable** by stating its **type** and **name** before using it.
- A **variable name** can only be on the left hand side of an **=** (*the assignment operator*) in which case it refers to the storage location

```
number = 4;           // change the value at variable number's storage location to 4
```

- A **variable name** in an expression means retrieve the **value** of the variable.

```
System.out.println( number ); // retrieve the value at variable number's storage location i.e. 4
```


Basic Concepts

Expressions and Statements

- An **expression** has **variables**, **operators**, **object creation** and **method invocations**.
 - Expressions evaluate to a single value of a resultant data type
 - The data type of the value after evaluation depends on the data types of the elements.
 - Expressions are constructed according to the syntax of the language.

```
4 * 4           // 16 of type int
4 * 1.5         // 6.0 of type double
2 * 4 + 8 / 6 - 10 % 3 // 8 of type int
```

- A **statement** forms a complete unit of execution. There are three types:
 1. **Declaration statements:** to declare variables and maybe initialise them

```
int result;           // declaration statement
int x = 4;            // declaration statement with initialisation
```

2. **Expression statements:** an expression terminated with a semicolon

```
result = 2 * x + 8 / 6 - 10 % 3; // assignment statement
System.out.println( result );    // method invocation statement
Circle circle1 = new Circle();   // object creation statement
```

3. **Control flow statements:** regulate the order in which statements get executed

Basic Concepts

Blocks/Compound Statements and the Empty Statement

- Normally methods have a sequence of **declaration** and **expression statements**.
- Statements can be grouped into a **block of statements**.
- A **block of statements**, aka a **compound statement** is a group of **zero** or more statements between left and right curly braces **{ }**

- Effectively the statements are packaged together

```
{  
    int x;           // x is local to this block, and exists from where it is  
    x = 4;           //      declared until the end of the containing block  
    result = x * x;  
}                   // x no longer exists after this end of block }
```

- When a program is executing, any variable created within a block is destroyed when the end of that block is reached.
- The statements within the **block** should be indented.
- The **Empty statement**
 - A **;** by itself indicates the **empty statement** (it does nothing)
- A **block** or **empty statement** can be used anywhere a single statement can.

Basic Concepts

Sequential structures and Control flow statements

- **Sequential structure**

- Programs normally execute sequentially
- Unless otherwise instructed, the computer executes Java statements one after the other in the order in which they are written (top to bottom).
- **Declaration statements** and **Expression statements** in a program execute one after the other in the order in which they are written.
- This **Sequential structure** is built into Java and other programming languages.

- **Transfer of control**

- Often the next statement we need to execute is not necessarily the next one in sequence.
- **Bohm and Jacopini** deduced that all programs can be written in terms of only three control structures: the **sequence structure**, the **selection structure** and the **repetition structure**.
- The Java **Control flow statements**: determine the order in which statements are executed and allow **selection** and **repetition**.

- **In summary, Java supports three different types of statements:**

- **Declaration statements**: declare variables.
- **Expression statements**: change values of variables, create objects, call methods.
- **Control flow statements**: determine the order that statements are executed.

Control flow statements

Selection Statements

Java has three **Selection Statements**

1. **if statement**

- Performs an action, if a condition is **true**; skips it, if **false**.
- It is a **Single-selection statement**
- It selects or ignores a **single** action or group of actions.

2. **if...else statement**

- Performs an action if a condition is **true** and performs a different action if the condition is **false**.
- It is a **Double-selection statement**
- It selects between two different actions or groups of actions.

3. **switch statement**

- Performs one of several actions, based on the value of an **expression**.
- It is a **Multiple-selection statement**
- It generally selects among many different actions (or groups of actions).

Control flow statements

Repetition Statements

- **Repetition Statements** are aka **looping statements**
 - A **loop** is a sequence of instructions that is continually repeated until a certain condition is met.
 - Loops are used e.g. to add numbers, repeatedly call methods, process a list, ...
- Java has three **Repetition Statements**
 1. **while**
 - Performs an action in the while body zero or more times based on a condition.
 - Perform statements repeatedly while a **loop-continuation condition** remains true.
 2. **do ... while**
 - Performs an action in its body one or more times based on a condition.
 3. **for**
 - Similar to the while statement but uses very different syntax.

Program Formation and Modelling control statements

- Every program is formed by using the sequence structure and combining the various statements as appropriate for the algorithm being implemented.

Declaration statements + Expression statements + Control-flow statements

Control-flow statements $\Rightarrow$ Selection + Repetition statements (three types each)

- Rest of this section focuses on explaining the **Control-flow statements**
 - Each **Control-flow statement** can be modelled in different ways.
 - Each student learns differently, so explore various model and use what works for you to build up a semantic model of what's going on within each statement.
 - Here we will model using pseudocode and using UML activity diagram extracts.
- UML activity diagram to illustrate **Control-flow statements**
 - Will show as **single-entry/single-exit control statements**
 - **Initial state** and a **final state** represent its entry point and exit point, respectively.
 - However, **Control-flow statements** can be stacked and nested.
 - Control-statement stacking - connect the exit point of one to the entry point of the next.
 - Control-statement nesting - a control statement inside another.

if

Single-Selection Statement

Pseudocode

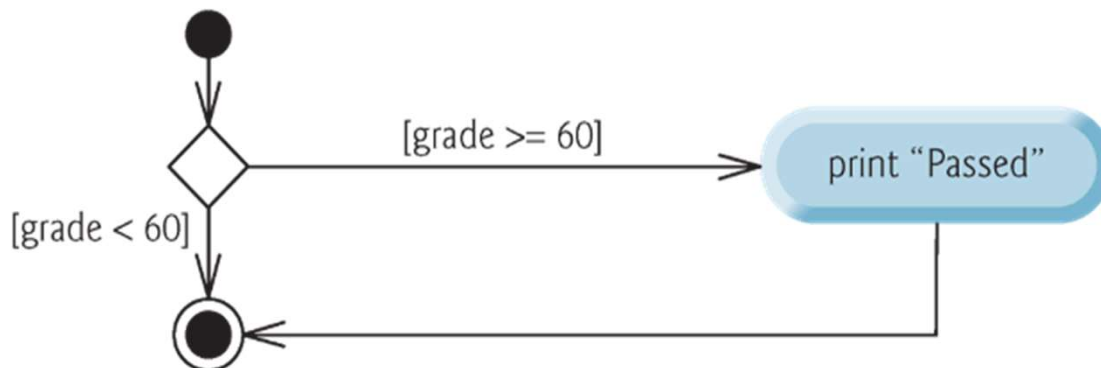
*If student's grade is greater than or equal to 60
Print "Passed"*

Java code

// assume a variable grade exists

```
if ( grade >= 60 )  
    System.out.println( "Passed" );
```

UML Activity Diagram



- ▶ If the condition is false, the Print statement is ignored, and the next pseudocode statement in order is performed.
- ▶ Indentation is Optional, but recommended to emphasise the inherent structure

**if (condition)
statement**

Specify an action e.g. **statement** to perform when the **condition** is true
() are required.

- ▶ Diamond, or **decision symbol**, indicates that a decision is to be made.
- ▶ Workflow continues along a path determined by the symbol's **guard conditions**, which can be true or false.
- ▶ Each transition arrow emerging from a decision symbol has a guard condition (in square brackets next to the arrow).

if else

Double-Selection Statement

Pseudocode

*If student's grade is greater than or equal to 60
Print "Passed"*

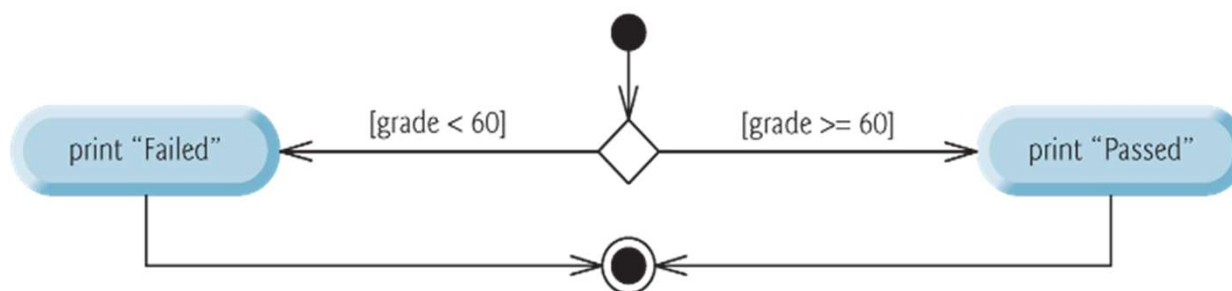
*Else
Print "Failed"*

Java code

// assume a variable grade exists

```
if ( grade >= 60 )  
    System.out.println( "Passed" );  
else  
    System.out.println( "Failed" );
```

UML Activity Diagram



- ▶ If the condition is false, the Print statement is ignored, and the next pseudocode statement in order is performed.

```
if ( condition )  
    statement_true  
else  
    statement_false
```

specify an action e.g. **statement_true** to perform when the **condition** is true and a different action **statement_false** when the **condition** is false. () are required.

- ▶ See [if](#) for UML explanation
- ▶ The major difference is that if the **guard condition** `[grade<60]` is true then the **action state** `print "Failed"` will be executed.

if and if else

Exercise 1

- Write a *code extract* that uses an **if** statement to check an integer **number** for *odd/even* and for *negative/positive*?

Solution Approach:

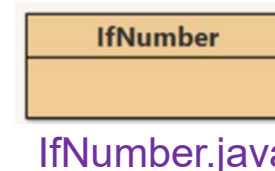
1. First, write out the code extract on paper

```
int number;           // Assume a variable number exists.
```

```
if (...
```

2. Later, use your IDE to code and check a complete solution in a **main** method (within a **class**)

3. A possible solution follows in and is available in



if and if else

IfNumber

IfNumber.java

```
import java.util.Scanner;
public class IfNumber
{
    public static void main(String[] args)
    {
        Scanner keyboard = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int number = keyboard.nextInt();

        System.out.printf("Number entered %d is ", number);

        if (number >= 0) // if (condition evaluates to true)
            System.out.print("Positive"); // execute the statement

        if (number < 0) // Similarly, But could use else instead
            System.out.print("Negative"); //

        if (number%2 == 0) // if (condition evaluates to true)
            System.out.printf(" Even\n", number); // execute the statement
        else // else
            System.out.printf(" Odd\n", number); // execute the statement
    }
}
```

Enter an integer: 5

Number entered 5 is Positive Odd

Enter an integer: -12

Number entered -12 is Negative Even

Conditional operator

? :

- Conditional operator (?:)
 - Can be used as a shorthand if...else.
 - Java's only **ternary** operator (takes three operands)
- Operands and ?: form a **conditional expression**

operand_1 ? operand_2 : operand_3

- **operand_1** must be a **boolean expression** and either evaluate to true or false
- **operand_2** is evaluated if the value of **operand_1** is true
- **operand_3** is evaluated if the value of **operand_1** is false

Example:

```
System.out.println( grade >= 60 ? "Passed" : "Failed" );
```

- The `println` argument expression evaluates to the string "Passed" if the boolean expression `grade >= 60` is true and to the string "Failed" if the boolean expression `grade >= 60` is false.

Conditional operator

? :

- ? : and its three operands form a conditional expression
`operand_1 ? operand_2 : operand_3`
- The entire conditional expression evaluates to
`operand_2` if `operand_1` is `true`, `operand_3` if `operand_1` is `false`

Example

```
{  
    int num1 = 3;  
    int num2 = 5;  
  
    int maximum;  
    maximum = (num1 > num2) ? num1 : num2;  
  
    // is equivalent to  
  
    if (num1 > num2)  
        maximum = num1;  
    else  
        maximum = num2;  
}
```

**Conditional expressions
are widely used but are
often more difficult to
read!**

Use with caution!

if else

nested statements

- Can nest multiple if cases by placing **if...else** statements inside other **if...else** statements to create **nested if...else statements**.

Pseudocode:

*If student's grade is greater than or equal to 90
 Print "A"*

Else

*If student's grade is greater than or equal to 80
 Print "B"*

Else

*If student's grade is greater than or equal to 70
 Print "C"*

Else

*If student's grade is greater than or equal to 60
 Print "D"*

Else

Print "F"

if else nested Java code

Java code

Assume a
variable:
studentGrade
exists holding
the student's
grade.

```
if ( studentGrade >= 90 )
    System.out.println( "A" );
else
    if ( studentGrade >= 80 )
        System.out.println( "B" );
    else
        if ( studentGrade >= 70 )
            System.out.println( "C" );
        else
            if ( studentGrade >= 60 )
                System.out.println( "D" );
            else
                System.out.println( "F" );
```

- If **studentGrade** is 93 (i.e. ≥ 90) then the first four conditions will be true
- But only the statement in the **if** part of the first **if...else** statement will execute.
- After that, the **else** part of the “outermost” **if...else** statement is skipped, which includes all the inner if statements

if else

nested Java code compact form

Java code

- Most Java programmers prefer to write the preceding nested `if...else` statement with subsequent `if` conditions directly following the `else`.
- The two forms are identical except for the spacing and indentation, which the compiler ignores.

```
if ( studentGrade >= 90 )
    System.out.println( "A" );
else if ( studentGrade >= 80 )
    System.out.println( "B" );
else if ( studentGrade >= 70 )
    System.out.println( "C" );
else if ( studentGrade >= 60 )
    System.out.println( "D" );
else
    System.out.println( "F" );
```

if else

nested if statements and dangling else

- Asked to develop code that checks and reports:
if x and y are both > 5
if x is <= 5
- The Java compiler always associates an **else** with the immediately preceding **if** unless told to do otherwise by the placement of braces { }.
- Referred to as the **dangling-else problem**.
- Resolve by using { } or alternative **if** statements

1.

```
if ( x > 5 )           // Code doesn't do as we expect
    if ( y > 5 )
        System.out.println( "x and y are > 5" );
else
    System.out.println( "x is <= 5" );
```

2.

```
if ( x > 5 )           // Compiler interprets above as:
    if ( y > 5 )
        System.out.println( "x and y are > 5" );
    else
        System.out.println( "x is <= 5" );
```

3.

```
if ( x > 5 )           // Corrected to do as intended:
{
    if ( y > 5 )
        System.out.println( "x and y are > 5" );
}
else
    System.out.println( "x is <= 5" );
```

Try values of x = 3, y = 7, only **3. if** actually prints "x is <= 5"

Empty statement, single statement, block of statements if examples

- The **statement** in a control flow statement can be an **empty statement**, a **single statement** or a **block of statements**.

- Empty statement ;

```
if ( grade >= 60 )  
    ;           // Do nothing
```

- Single statement

```
if ( grade >= 60 )  
    System.out.println( "Passed" );
```

- Block of statements ein { }

```
if ( grade >= 60 )  
{  
    System.out.println( "Grade is " + grade );  
    System.out.println( "Passed" );  
}
```

```
if ( condition )  
    statement
```

```
if ( condition )  
    statement_true  
else  
    statement_false
```

Many coding
guidelines require
blocks to be used
in all cases, { } is
the empty block

if and if else

Exercise 2

IfExamples

IfExamples.java

1. Write a code extract to find the maximum of two **integer** variables a and b
2. Write code to find the maximum of three **integer** variables a, b and c
3. Write a code extract that prints “Good scores” if the values of two **double** variables grade1 and grade2 are greater than 80.
4. Modify above to print “Better luck next time” if either score is less than or equal to 80.

Solution Approach:

1. First, write out the code extract on paper
2. Later, check by coding a complete solution in a **main** method.
3. Complete solutions are available in the class

IfExamples

if and if else

Partial Solution

2. Write code to find the maximum of three integer variables **a**, **b** and **c**

Code Extract, assume variables **a**, **b**, **c** and **max** exist.

```
if ( a > b && a > c )
    max = a;
else if ( b > c )      // Note if ( a > b && b > c ) doesn't work try a=5 b=5 c=2
    max = b;
else
    max = c;

// More efficient (and simpler) way which can easily be extended for more variables
max = a;                // Let's assume a is largest
if ( b > max )           // Now if b is larger than max (a)
    max = b;            // Set max to b
if ( c > max )           // Now if c is larger than max (of a and b)
    max = c;            // Set max to c
System.out.printf("max(%d, %d, %d) is %d\n", a, b, c, max );
```

Static methods

- A method is declared **static** in a class where the method does not access any **instance** variables. Include the **static** keyword after the access keyword.
- Static methods may be **void** or return a **primitive** or **reference** type:

```
public static void methodName( optionalParameters )
{
    statements
}

public static returnType methodName( optionalParameters )
{
    statements
    return expression;
}
```

- Static methods can be called directly without creating an object of the class.

methodName(optionalArguments); // if in the same class

ClassName.methodName(optionalArguments); // if in class **ClassName**

Boolean methods

IsMethods

IsMethods.java

- A **boolean method** returns a **boolean** value either **true** or **false**.
- The method identifier/name is often prefixed, as for **boolean** variables, with the verb **is**:
 - **isWhatYouAreCheckingFor()**
 - **isEven()** or **isNumberEven()**
 - **isEmpty()**
 - **isComplete()**
- Example Boolean methods
 - **isEven**: check a number for even or odd
 - **isPositive**: check a number for positive or negative
 - Full solution in the class

IsMethods

*Note the **Javadoc** method comments*

```
/**
 * Check if a number is even or odd
 * @param n number to check
 * @return true if even, false if odd
 */
public static boolean isEven(int n)
{
    return n%2==0;
}

/**
 * Check if a number is positive
 * @param n number to check
 * @return true if positive, false if not
 */
public static boolean isPositive(int n)
{
    return n>0;
}
```


Indentation, Syntax and Logic Errors

- Use indentation in bodies of control flow statements, such as **if else**
- Consider using blocks of statements for the bodies, i.e. always enclose in **{ }**
- **Syntax errors** are caught by the compiler and must be resolved e.g.
 - Forgetting one or both of the **{ }** that delimit a block
 - Using an **=** instead of **==** for equality test in condition (may result in a cryptic Error)
- A **logic error** has its effect at execution time.
 - A **fatal logic error** causes a program to fail and terminate prematurely.
 - A **nonfatal logic error** allows a program to continue executing but causes it to produce incorrect results. Such errors can be more catastrophic.
 - Leaving out the **{ }** in an if block when a block of statements required.
 - A semicolon after **if (condition) ;** always executes the following **statement**

Be warned! These type of relatively common errors if not found by testing can have major consequences on a running system

Repetition statements

- **Repetition statements** are used to repeat actions
 - The repetition happens while a **condition** remains true.
 - The **condition** is either checked before or after the action is performed.
 - The **repetition statement**'s body may be a single statement or a block.
 - The **condition** can be counter based or sentinel (flag) based.
- Eventually, the **condition** must become **false** for repetition to terminate.
 - When the repetition terminates, the first statement after it will execute.
 - The body of the repetition statement must have an action that eventually will cause the condition to become false, otherwise we have an **infinite loop**.

```
While there are more compilation errors           // Counter based  
    Review the next listed error and resolve it.
```

```
While the user selects to continue playing music // Sentinel based  
    Play a random song from the collection
```

- The **while** iteratively checks the **condition** and performs the statement in its body while the condition is true.

while

Repetition Statement

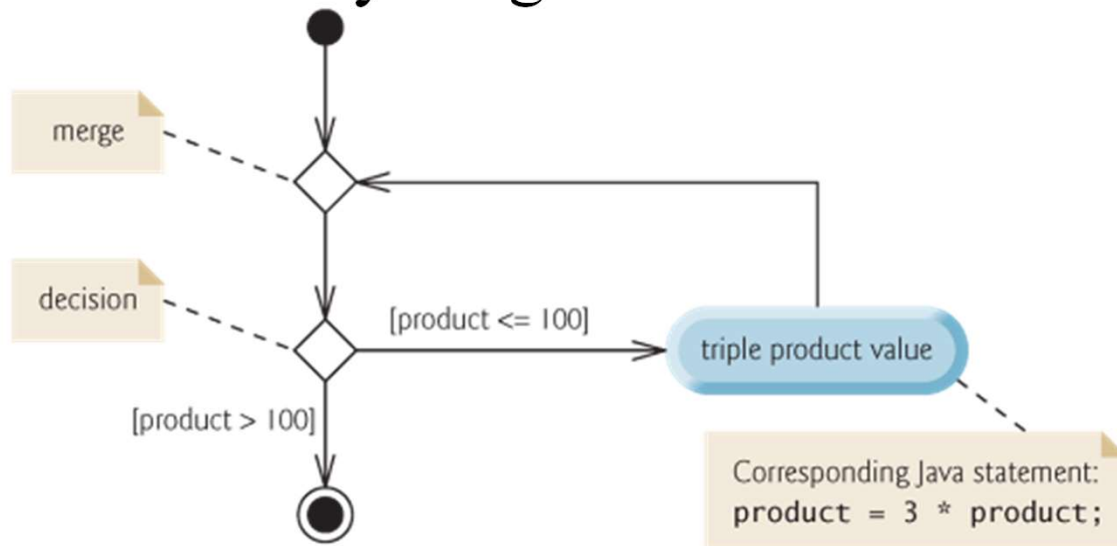
Pseudocode

Initial variable product to 1
while product is less than or equal to 100
triple product value

Java code

```
int product = 1;  
while ( product <= 100 )  
    product = 3 * product;
```

UML Activity Diagram



while (condition) statement

The **while** iteratively checks the **condition** and executes the **statement** in its body while the **condition** is true.

Specify the action e.g. **statement** to perform while the **condition** is true. () are required around condition.

- ▶ A **decision** symbol has one transition arrow pointing to the diamond and two or more pointing out from it to indicate possible transitions from that point. Each transition arrow pointing out of a decision symbol has a guard condition next to it.
- ▶ A **merge** symbol has two or more transition arrows pointing to the diamond and only one pointing from the diamond, to indicate multiple activity flows merging to continue the activity. There are no guard conditions.

while

Repetition Statement

- Example of the Java **while** repetition statement:

```
/*  
 * Find and print the first power of 3 larger than 100  
 */  
{  
    int product = 1;  
    while ( product <= 100 )  
        product = 3 * product;  
    System.out.println("Product of 3 > 100 is " + product);  
}
```

- Each iteration multiplies **product** by 3, so **product** takes on the values 9, 27, 81 and 243 successively. **repetition**
- When variable **product** becomes 243, the **while**-statement condition—**product <= 100**—becomes false.
- Repetition terminates. The final value of **product** is 243.
- Program execution continues with the next statement after the **while** statement.

while

Example: print numbers 1 to 10

Consider, printing the numbers from 1 to 10, one per line.

1. Create a **counter** variable (aka a **control variable**)
2. Initialize the variable to the start of range
3. Use a **while** with a **loop condition** on the variable to become **false** at the range end
4. Increment (add 1 to) the **counter** in the loop body

```
int counter = 1;
while (counter <= 10)
{
    System.out.println( counter );
    counter = counter + 1;
}
```

Output

1
2
3
4
5
6
7
8
9
10

The **while** statement's body may be: the empty statement, a single statement or a block of statements enclosed in { }. Use a block { }.

while

Counter-controlled repetition

WhileCounter

WhileCounter.java

- Many repetitions of tasks require the use of a **counter** to control the repetition.
- First decide the inclusive range over which you wish to count e.g. [1, 10]
- Then add a **control variable** e.g. **counter**. Often, variable **i** is used.
 - Declare and initialise the **control variable**, before the loop begins, in most cases 0 or 1.
 - Determine the **loop continuation condition** that determines if the loop should continue,
 - **Increment** or **decrement** the **control variable** in the loop after each the loop task.

```
public class WhileCounter
{
    public static void main(String[] args)
    {
        int counter = 1;
        while (counter <= 10)
        {
            System.out.println( counter );
            counter = counter + 1;
        }
    }
}
```

Declare and initialise **control variable**

Loop continuation condition

Loop task i.e. print counter

Increment **control variable**

while

More Examples

WhileExamples

WhileExamples.java

Write code snippets to:

1. Print the numbers from 1 to 10, one per line.
2. Print the numbers from 5 10 15 20 ... 50, on same line.
3. Print the numbers from 40 to 100 that are divisible by 7
4. Print the numbers from 11100 to 11200 divisible by 7
5. Sum the even numbers from 1 to 100
6. Write out the points on the line $y = 2x + 1.5$, for $x=0.0$ to 4.0 in steps of 0.5

while

More Examples: sum

```
/*
 * 5. Write code to sum the even numbers from 1 to 100
 */
{
    System.out.print( "\nSum the even numbers from 1 to 100, " );
    int sum = 0;
    int evenNumber = 2;           // First even number
    while ( evenNumber <= 100 )
    {
        sum += evenNumber;        // sum = sum + evenNumber;
        evenNumber += 2;          // Next even number
    }
    System.out.println( "the sum is " + sum );
}
```

while

More Examples: line points

```
/*
 * 6. Write out the points on the line  $y = 2x + 1.5$ , for  $x=0.0$  to  $4.0$  in steps of  $0.5$ 
 */
{
    System.out.println("\nPoints on a line  $y = 2x + 1.5$ , for  $x=0.0$  ..  $4.0$ , stepsize  $0.5$ ");

    int point = 0;           // Use integer variable for counting, avoid real numbers
    while (point <= 8)
    {
        double x = point * 0.5;
        double y = 2*x + 1.5;
        System.out.printf("(%.1f,%.1f) ", x, y);
        point++;
    }
    System.out.println();
}
```

while

sum numbers

WhileSum

WhileSum.java 1 of 2

- The **WhileSum** class has two methods: **int sum(int n)** **void main(String[] args)**
- The method **sum** sums the numbers from **1** to the value of the parameter **n**

```
/**
 * Sum the numbers from 1 to n
 * @param n      the upper limit (inclusive) to sum to
 * @return the sum
 */
public static int sum(int n)
{
    int sum = 0; // Local variable to hold sum

    int i = 1;   // Local counter/loop variable
    while (i <= n)
    {
        sum += i;           // sum = sum + i;
        i += 1;             // i = i+1, i++, ++i, do the same
    }
    return sum;
}
```

while

sum numbers

WhileSum

WhileSum.java 2 of 2

- The **WhileSum main** method calculates and displays the partial sums from 1 to 10, and calls the **sum** method with arguments 20 and 10000.

```
public class WhileSum
{
    public static void main(String[] args)
    {
        int sum = 0;           // variable used to sum the numbers

        System.out.println("Partial sums from 1 to 10");
        int i = 1;             // Loop counter variable (local)
        while (i <= 10)
        {
            sum = sum + i;
            System.out.printf("%3d %3d\n", i , sum);    // Display partial sums
            i = i + 1;           // i++;
        }

        // Call method sum below to do summation
        System.out.println( "    sum(20) is " + sum(20) );
        System.out.println( " sum(10000) is " + sum(10000) );
    } // end main method
}
```

Partial sums from 1 to 10	
1	1
2	3
3	6
4	10
5	15
6	21
7	28
8	36
9	45
10	55
sum(20) is 210	
sum(10000) is 50005000	

while

Calculate Average Marks

Determine the average score in a test or quiz.

- A class of **N** students took a test/quiz.
- The marks are integers in the range 0 to 100.
- The marks to be entered for this test are already available.
- Hence,
 - We have to ask the user for **N** the number of students/marks
 - And then get the **N** marks
 - Finally, calculate the average, equal to the sum of the marks divided by **N**.
- Best to start by writing out pseudocode for the solution (algorithm).
- Use **counter-controlled repetition** to input the grades one at a time.
 - A variable called a counter (or control variable) controls the number of times a set of statements will execute.
 - Counter-controlled repetition is often called definite repetition, because the number of repetitions is known before the loop begins executing.

while

Calculate Average Marks

Initial Pseudocode

- Most frequently to solve a problem we use **top-down, stepwise refinement**.
- Start with a **top level description** of what you want to do.

Find the average mark of a test/quiz.

- Refine to a **First level refinement** sequence of tasks.
 1. *Describe the program to the user*
 2. *Identify and initialise variables*
 3. *Input, count and sum the marks*
 4. *Calculate the average and print it*
- Then refine each of these tasks into a **Second level refinement** and so on
- Until the end result is Pseudocode where the Java code is readily writeable.

while

Calculate Average Marks

Detailed Pseudocode

1. **Print** "Program to find the average mark of a test/quiz"
2. **Print** "How many student/marks are there: "
3. **Input** number and store in variable **numberMarks** or simply **Input** **numberMarks**
4. **Print** "Enter " + **numberMarks** + " numbers to find the average"
5. **Create** variable **sumMarks** and initialise to 0
6. **Create** counter variable **countMarks** and initialise to 1
7. **While** **countMarks** is less than or equal to **numberMarks**
 8. **Print** "Enter Marks"
 9. **Input** marks and store in variable **mark**
 10. **Add** **mark** to **sumMarks**
 11. **Add one** to **countMarks**
12. **endWhile**
13. **Create** variable **average** and calculate the average **sumMarks/numberMarks**
14. **Print** "Sum of the " + **numberMarks** + " is " + **sumMarks**
15. **Print** "Average mark is " + **average**

while

Calculate Average Marks

```
public class WhileAverageMark
{
    public static void main(String[] args)
    {
        // Often programmers declare all variables at the beginning
        int numberMarks;           // number of marks to process
        int sumMarks;              // sum of marks entered by user
        int countMarks;           // count number of the mark to be entered
        int mark;                 // mark entered by user

        Scanner keyboard = new Scanner( System.in );

        System.out.println("Program to find the average mark of a test/quiz");

        System.out.print("How many student/marks are there: ");
        numberMarks = keyboard.nextInt();    // read number of marks from user
    }
}
```

while

Calculate Average Marks

```
System.out.printf("\nEnter %d numbers to find the average", numberMarks);

sumMarks = 0;           // initialize sum of marks
countMarks = 1;         // initialize loop counter
while ( countMarks <= numberMarks )    // Iterate
{
    System.out.printf( "Enter mark[%d]: ", countMarks );
    mark = keyboard.nextInt(); // read mark from user
    sumMarks += mark;          // add mark to sum
    countMarks++;              // increment counter by 1
} // end while

int average = sumMarks/numberMarks; // integer division yields integer result
System.out.printf( "\nSum of the %d marks is %d\n", numberMarks, sumMarks );
System.out.printf( "Average mark is %d\n", average );
}
```

while

Calculate Average Marks

Execution Example

```
Program to find the average mark of a test/quiz  
How many student/marks are there: 5
```

```
Enter 5 numbers to find the average
```

```
Enter mark[1]: 80
```

```
Enter mark[2]: 72
```

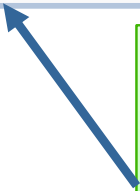
```
Enter mark[3]: 65
```

```
Enter mark[4]: 48
```

```
Enter mark[5]: 93
```

```
Sum of the 5 marks is 358
```

```
Average mark is 71
```



Note the integer result $358/5 = 71$.
Dividing two integers results in an integer,
and it truncates as opposed to rounding
the fractional part as $358 \div 5 = 71.6$

while

Calculate Average Marks: Local variables

Local variables

```
public class WhileAverageMark
{
    public static void main(String[] args)
    {
        int numberMarks;
        int sumMarks;
        int countMarks;
        int mark;
    }
}
```

- Recall, that **variables** declared in a method body are called **local variables** and can be used only from the line where declared to the closing right brace of the block.
- The declaration of a **local variable** must appear before it is used in a method.
- A **local variable** cannot be accessed outside the method in which it is declared.
- A **local variable** must be initialized before first usage,
 - sum variables to 0, product variables to 1, counters to 0 or 1 depending on usage.
- Many programmers declare **local variables** at the beginning of a method.
- A **local variable** name should be self explanatory; no need for explanatory comment

while

Calculate Average Marks using a Sentinel

WhileSentinel.java 1 of 2

- Calculate average for an arbitrary number of students using a sentinel

```
public class WhileSentinel
{
    private static Scanner keyboard = new Scanner( System.in );

    public static void main(String[] args)
    {
        System.out.println("Program to find the average mark of a test/quiz");
        System.out.println("Enter marks [-1 when no more marks]");

        int sumMarks = 0;
        int countMarks = 0;           // initialize to 0 as no marks so far
        int mark = 0;
        while ( mark != -1 )
        {
            System.out.printf( "Enter mark[%d]: ", countMarks + 1 );
            mark = keyboard.nextInt(); // read mark from user
            if (mark != -1)
            {
                sumMarks += mark;
                countMarks++;
            }
        }
    }
}
```

Use a sentinel of -1 for mark in the **while** condition to control the **while** loop.

Note: Select a **sentinel** value that doesn't conflict with a valid input value.

while

Calculate Average Marks using a Sentinel

WhileSentinel.java 2 of 2

```
if (countMarks > 0)
{
    int average = sumMarks/countMarks;
    System.out.printf( "\nSum of the %d marks is %d\n", countMarks, sumMarks );
    System.out.printf( "Average mark is %d\n", average );
}
else
    System.out.println(">>> No marks were entered");
}
```

Program to find the average mark of a test/quiz

Enter marks [-1 when no more marks]

Enter mark[1]: 80

Enter mark[2]: 72

Enter mark[3]: 65

Enter mark[4]: 48

Enter mark[5]: 93

Enter mark[6]: -1

Using a sentinel of -1 to indicate completion

Sum of the 5 marks is 358

Average mark is 71

Increment and Decrement Operators

++ --

- Unary **increment operator**, **++**, adds one to its operand
- Unary **decrement operator**, **--**, subtracts one from its operand
- Can prefix (*place before*) or post fix (*place after*) to a variable
- The position only impacts what happens when the variable is used in an expression.
 - If prefixed it is a **prefix increment/decrement operator**
 - If postfixed it is a **postfix increment/decrement operator**
- We use terms **preincrement**, **predecrement**, **postincrement**, **postdecrement**.

Operator	Example	Explanation
++ Prefix increment	++counter	Increment/Decrement counter by 1, then use the new value of counter in the expression where the counter exists.
-- Prefix decrement	--counter	
Postfix increment ++	counter++	Use the current value of counter in the expression where the counter exists, then Increment/Decrement counter by 1.
Postfix decrement --	counter--	

Compound Assignment Operators

`+=` `-=` `*=` `/=` ...

- Statements where a variable occurs on both sides of the assignment operator may be abbreviated using a **Compound Assignment Operator**

variable = variable operator expression `result = result + 10;`

can be rewritten as

variable operator = expression `result += 10;`

Operator	Name	Example	Equivalent to
<code>=</code>	Assignment operator	<code>result = 0;</code>	
<code>+=</code>	Addition Assignment operator	<code>result += 4;</code>	<code>result = result + 4;</code>
<code>-=</code>	Subtraction Assignment operator	<code>result -= 4;</code>	<code>result = result - 4;</code>
<code>*=</code>	Multiplication Assignment operator	<code>result *= 4;</code>	<code>result = result * 4;</code>
<code>/=</code>	Division Assignment operator	<code>result /= 4;</code>	<code>result = result / 4;</code>
<code>%=</code>	Remainder Assignment operator	<code>result %= 4;</code>	<code>result = result % 4;</code>
<code>...</code>	Many others.		

Cast Operator ()

- The **Cast operator ()** performs **explicit conversion** or **type cast**.
 - Cast operator formed by placing parentheses around the name of a type.
 - The unary cast operator **(int)** creates a temporary integer copy of its operand.

```
double result = 12.59;
```

```
int resultApprox = (int) result; // value stored in result is unchanged
```

- Binary operators can only be evaluated for identical operand types
 - **Promotion** (or implicit conversion) is performed on operands.
 - In an expression containing values of the types **int** and **double**, the **int** values are promoted to **double** values for use in the expression.
- Cast operators are available for any type.
 - The operator is a unary operator (it only takes only one operand).
 - Other unary operators: **unary plus** (+) and **unary minus** (–) operators.
 - Cast operators associate from right to left; same precedence as unary operators

Refer to Java Operator Precedence chart for full details

Logical Operators

- Many of the condition expressions met so far are simple e.g. **counter <= 10**
- Often need to form more complex conditions through combining simple conditions
- There is a set of **Conditional** and **Boolean** operators, all of which require **boolean** operands and yield a **boolean** result (recall **boolean** is either **true** or **false**).
- In the examples below **expr1** and **expr2** are **boolean expressions**.
- A **boolean** expression, evaluates to **true** or **false**.

Operator	Name	Example	Equivalent to
!	Logical NOT	<code>!expr</code>	true if <code>expr</code> is false , and vice versa.
&&	conditional AND	<code>expr1 && expr2</code>	true if both <code>expr1</code> and <code>expr2</code> are true
	conditional OR	<code>expr1 expr2</code>	true if either <code>expr1</code> and <code>expr2</code> are true
&	boolean logical AND	<code>expr1 & expr2</code>	true if both <code>expr1</code> and <code>expr2</code> are true
	boolean logical inclusive OR	<code>expr1 expr2</code>	true if either <code>expr1</code> and <code>expr2</code> are true
^	boolean logical exclusive OR	<code>expr1 ^ expr2</code>	true if only <code>expr1</code> or <code>expr2</code> are true

Aside: the &, | and ^ operators are bitwise operators when the operands are integral.

Logical Operators

! Logical NOT operator

- The **Logical NOT ! Operator** is a unary operator that yields the negation of the **boolean** operand.

yields **true** if operand is **false**
yields **false** if operand is **true**.

- Unary: it operates on a single operand.
- A **boolean** operand value is either **true** or **false**
- In many cases the logical operator can be avoided by rewriting the boolean expression.

Assume variable **x** exists: **int x**;

```
if (!(x>10) )           // Same as if (x<=10)
    System.out.println("x is not > 10, i.e. x≤10");

boolean isTaskComplete = false;
if (!isTaskComplete)
    System.out.println("The task is not complete...");
```

Assume variable **x** exists: **int x**;

Truth Table Logical NOT operator !

expr	!expr
false	true
true	false

A truth table is a table used in boolean algebra which shows the truth-value of a compound expression for every truth-value of its component expressions.

Logical Operators

&& conditional AND operator

- The **conditional AND operator &&** returns **true** if both operands are **true** returns **false** otherwise.
- It 'short-cuts': if the first operand is **false** then the second operand is not evaluated.
- The **if** expression evaluates to true when $1 \leq x \leq 10$.

Assume variable **x** exists: **int x;**

```
if (x >= 1 && x <= 10)
    System.out.println("x is within range [1,10]");
else
    System.out.println("x is outside range [1,10]");
```

Truth Table
Conditional AND operator &&

expr1	expr2	expr1 && expr2
false	false	false
false	true	false
true	false	false
true	true	true

Truth tables are widely used to understand and simplify **boolean** expressions.

Logical Operators

|| conditional OR operator

- The **conditional OR operator ||** returns **true** if either of its operands are **true** returns **false** otherwise.
- It 'short-cuts': if the first operand is **true** then the second operand is not evaluated.
- The **if** expression evaluates to true when **x<1 or x>10**:

Assume variable **x** exists: **int x**;

```
if (x < 1 || x > 10)
    System.out.println("x is outside range [1,10]");
else
    System.out.println("x is within range [1,10]");
```

Truth Table
Conditional OR operator ||

expr1	expr2	expr1 expr2
false	false	false
false	true	true
true	false	true
true	true	true

Also, consider using **Karnaugh maps** to simplify boolean expressions.

Logical Operators

^ boolean logical exclusive OR (XOR) operator

- The **boolean exclusive OR operator ^** returns **true** only if one of its operands is **true** and returns **false** otherwise.
- It must evaluate both operands.
- The **if** expression below evaluates to **true** if the algebraic sign of x and y are different:

Assume variable **x, y** exists: **int x; int y;**

```
if (x >= 0 ^ y >= 0)
    System.out.println("x and y have different signs, one +ve, one -ve");
else
    System.out.println("x and y have the same sign, both +ve or -ve");
```

Truth Table
Conditional OR operator ^

expr1	expr2	expr1 ^ expr2
false	false	false
false	true	true
true	false	true
true	true	false

XOR is less common, and should be used with caution.

Logical Operators

& boolean logical AND operator

- The **boolean logical AND operator &** returns **true** if both operands are **true** returns **false** otherwise.
 - Same result as Conditional AND except both operands are always evaluated.
- The **if** expression below evaluates to true when $1 \leq x \leq 10$.

Assume variable **x** exists: **int x;**

```
if (x >= 1 & x <= 10)
    System.out.println("x is within range [1,10]");
else
    System.out.println("x is outside range [1,10]");
```

Truth Table
boolean logical AND operator &

expr1	expr2	expr1 & expr2
false	false	false
false	true	false
true	false	false
true	true	true

The conditional **&&** is the more commonly used **AND**.

The boolean logical **&** operator is not available in other languages: e.g. C or C++.

Logical Operators

boolean logical inclusive OR operator

- The boolean logical inclusive OR operator `|` returns **true** if either operand is **true** returns **false** otherwise.
 - Same result as Conditional OR except both operands are always evaluated.
- The if expression evaluates to true when `x < 1` or `x > 10`:

Assume variable **x** exists: `int x;`

```
if (x < 1 | x > 10)
    System.out.println("x is outside range [1,10]");
else
    System.out.println("x is within range [1,10]");
```

Truth Table

boolean inclusive OR operator `|`

expr1	expr2	expr1 expr2
false	false	false
false	true	true
true	false	true
true	true	true

The Conditional `||` is the more commonly used OR.

The boolean `|` operator is not available in other languages e.g. C or C++.

for

Repetition Statement

```
for (initialisation; condition; increment)  
    statement
```

The **for** **initialisation** is executed once.

The **for** then iteratively checks the **for** **condition** and if **true** executes the **for** **statement** body followed by the **for** **increment**. Note the ; are required in **for(; ;)**

- The **for repetition** (control flow) statement is widely used as it improves code readability in particular for **counter controlled repetition**.
- Normally, the loop iteration and loop task are separated into **for header** and **for body**.
 - The **for header** contains all the information to control the repetition
 - The **for body statement** performs the loop task.
- To complete a **for**, decide the iteration range and the task to be performed.
 - Select a **control variable** e.g. **counter**. Often, variable **i** is used.
 - Declare and initialise the **control variable**, in the **for initialisation**, in most cases 0 or 1.
 - Determine the **for** loop continuation **condition** that determines if the loop should continue,
 - **Increment** or **decrement** the **control variable** in the **for increment** part.

for Repetition Statement

Pseudocode

*for number in the range 1 to 10
print number*

Java

**for (initialisation; condition; increment)
statement**

The **for initialisation** is executed once.

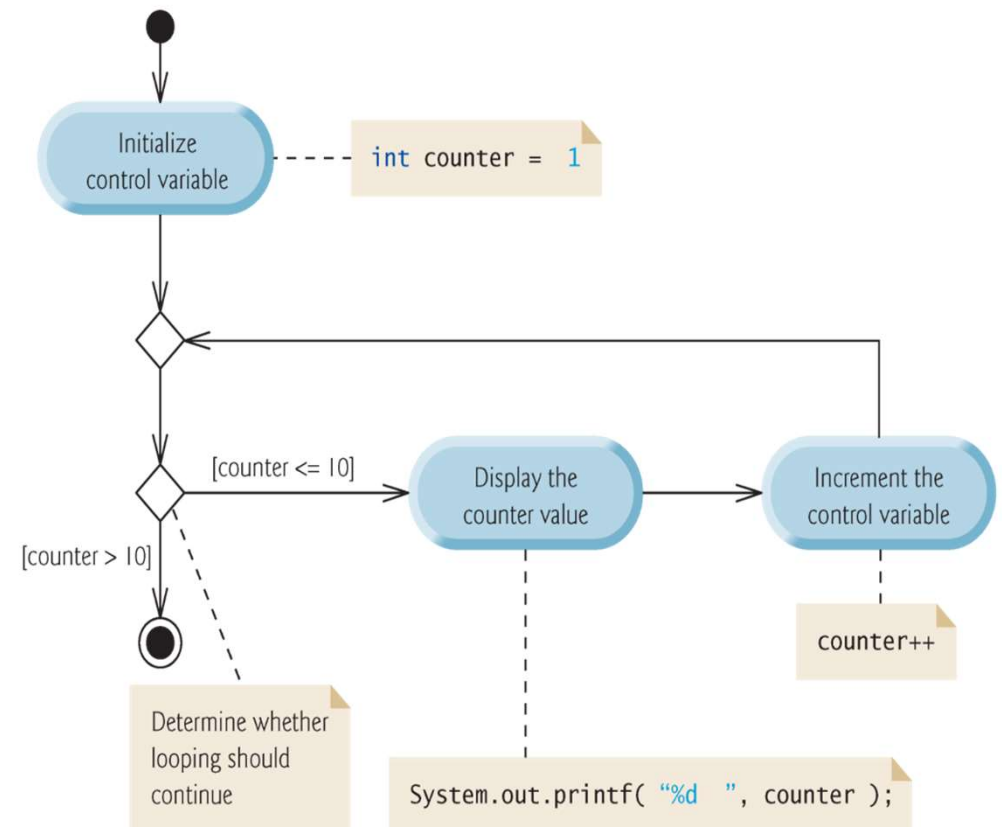
The **for** then iteratively checks the **for condition** and if true executes the **for statement** body followed by the **for increment**.

```
for(int counter = 1; counter <= 10; counter++)  
    System.out.printf( "%d ", counter );
```

Output

1 2 3 4 5 6 7 8 9 10

UML Activity Diagram



for

Example: print numbers 1 to 10

Pseudocode

for number in the range 1 to 10
print number

for (**initialisation**; **condition**; **increment**)
statement

- Define the iteration range **[1,10]** and the task to be performed **print number**.
- Then decide on a **control variable counter** normally of type **int**.

Declare and initialise the **control variable**, in the **for initialisation**

Determine the loop continuation **for condition** that determines if the loop should continue

Increment the **control variable** in the **for increment** part

```
for (int counter = 1; counter <= 10; counter++)  
    System.out.println( counter );
```

The **for** statement's body may be: the empty statement, a single statement or a block of statements enclosed in { }.

Output

1
2
3
4
5
6
7
8
9
10

for

Counter-controlled repetition

ForCounter

ForCounter.java

- In most cases the **for** statement is used for counter controlled repetition.

```
for (initialisation; condition; increment)  
    statement
```

- But a **for** statement can be represented with an equivalent **while** statement.

```
initialisation;  
while (condition)  
{  
    statement  
    increment;  
}
```

```
public class ForCounter  
{  
    public static void main(String[] args)  
    {  
        for (int counter = 1; counter <= 10; counter++)  
            System.out.println( counter );  
    }  
}
```

```
public class WhileCounter  
{  
    public static void main(String[] args)  
    {  
        int counter = 1;  
        while (counter <= 10)  
        {  
            System.out.println( counter );  
            counter = counter + 1;  
        }  
    }  
}
```

for

Code examples

```
// a) Vary the control variable from 1 to 100 in increments of 1.
```

```
for ( int i = 1; i <= 100; i++ )  
    ;
```

```
// b) Vary the control variable from 100 to 1 in decrements of 1.
```

```
for ( int i = 100; i >= 1; i-- )  
    ;
```

```
// c) Vary the control variable from 7 to 77 in increments of 7.
```

```
for ( int i = 7; i <= 77; i += 7 )  
    ;
```

```
// d) Vary the control variable from 20 to 2 in decrements of 2.
```

```
for ( int i = 20; i >= 2; i -= 2 )  
    ;
```

```
//e) Vary the control variable over the values 2, 5, 8, 11, 14, 17, 20.
```

```
for ( int i = 2; i <= 20; i += 3 )  
    ;
```

```
//f) Vary the control variable over 99, 88, 77, 66, 55, 44, 33, 22, 11, 0
```

```
for ( int i = 99; i >= 0; i -= 11 )  
    ;
```


for

Counter-controlled repetition

for (**initialisation**; **condition**; **increment**)
statement

- The **for initialisation**, **condition** and **increment** can contain arithmetic expressions
- The **for initialisation** can manipulate multiple variables.

```
for(int i = 1, j = 2; i * j <= 36; i++, j+=2)  
    System.out.printf( "(%d*%d) = %d \n", i, j, i*j );
```

- In many cases it can and should be simplified to a single **integer** variable.
- Simplify the **for** header above by observing the relationship between i and j
- The **for condition** must be a **boolean expression** and evaluate to **true** or **false**.
- The **for statement** body may be: the empty statement, a single statement or a block of statements enclosed in { }.
- **for(;;)** is equivalent to **while(true)**, repeats **for statement** forever (*until force stopped*).

for

Incrementing

for (**initialisation**; **condition**; **increment**)
statement

- The **for initialisation**, **condition** and **increment** can contain arithmetic expressions
- The **for increment** can use increment or compound assignments **operators**.

for increment Integer variable int i;	Assignment	Increment/ Decrement	Compound Assignment	Code Examples
Increment control variable by 1	i = i + 1	++i, i++	i += 1	for (int i=1; i<=10; i=i+1) for (int i=1; i<=10; i+=1) for (int i=1; i<=10; i++)
Decrement control variable by 1	i = i - 1	--i, i--	i -= 1	for (int i=1; i<=10; i=i-1) for (int i=1; i<=10; i-=1) for (int i=1; i<=10; i--)
Increment control variable by 2	i = i + 2		i += 2	for (int i=1; i<=10; i=i+2) for (int i=1; i<=10; i+=2)
Multiply control variable by 2	i = i * 2		i *= 2	for (int i=1; i<=10; i=i*2) for (int i=1; i<=10; i*=2)

Keep the **for statement control** simple.

When possible, count from 0 or 1 using increments of 1

for

sum numbers, and even and odd numbers

ForSum

ForSum.java 1 of 2

- ForSum has methods: `int sum(int n)`, `int sumEven(int n)`, `int sumOdd(int n)`

```
/**
 * Sum the numbers from 1 to n
 * @param n      the upper limit (inclusive) to sum to
 * @return       the sum
 */
public static int sum(int n)
{
    int sum = 0;           // Local variable to hold sum

    for( int i = 1; i <= n; i++ )
    {
        sum += i;          // sum = sum + i;
    }
    return sum;
}
```

```
/**
 * Sum the even numbers from 0 to n
 * @param n      the upper limit (inclusive) to sum to
 * @return       the sum
 */
public static int sumEven(int n)
{
    int sum = 0;           // Local variable to hold sum

    for( int i = 2; i <= n; i+=2 )
    {
        sum += i;          // sum = sum + i;
    }
    return sum;
}
```

- `sumOdd` is similar to `sumEven`

for

sum numbers, and even and odd numbers

ForSum

ForSum.java 2 of 2

```
public class ForSum
{
    public static void main(String[] args)
    {
        int sum = 0;           // variable used to sum the numbers

        System.out.println("Partial sums from 1 to 10");
        for (int i=1; i <= 10; i++)
        {
            sum = sum + i;
            System.out.printf("%3d %3d\n", i , sum);    // Display partial sums
        }

        System.out.println( "    sum(20) is " + sum(20) );
        System.out.println( " sum(10000) is " + sum(10000) );

        System.out.println( "sumEven(20) is " + sumEven(20) );
        System.out.println( " sumOdd(10) is " + sumOdd(10) );

    } // end main method
}
```

Class **ForSum** also has sum, sumEven and sumOdd methods...

```
Partial sums from 1 to 10
1  1
2  3
3  6
4 10
5 15
6 21
7 28
8 36
9 45
10 55

    sum(20) is 210
sum(10000) is 50005000
sumEven(20) is 110
sumOdd(10) is 25
```

Review ForExamples.java

for code snippet examples (similar to the **while** examples):

1. Print the numbers from 1 to 10, one per line.
2. Print the numbers from 5 10 15 20 ... 50, on same line.
3. Print the numbers from 40 to 100 that are divisible by 7
4. Print the numbers from 11100 to 11200 divisible by 7
5. Sum the even numbers from 1 to 100
6. Write out the points on the line $y = 2x + 1.5$, for $x=0.0$ to 4.0 in steps of 0.5

for

More Examples: line points

ForExamples.java

```
/*
 * 6. Write out the points on the line  $y = 2x + 1.5$ , for  $x=0.0$  to  $4.0$  in steps of  $0.5$ 
 */
{
    System.out.println( "\nPoints on a line  $y = 2x + 1.5$ , for  $x=0.0$  ..  $4.0$ , stepsize  $0.5$ ");

    for( int point = 0; point <= 8; point++)    // Count using integer variable
    {
        double x = point * 0.5;
        double y = 2*x + 1.5;
        System.out.printf("(%.1f,%.1f) ", x, y);
    }
    System.out.println();
}
```

- Compare with **while** equivalent in WhileExamples.java

```
int point = 0;
while (point <= 8)
{
    double x = point * 0.5;
    double y = 2*x + 1.5;
    System.out.printf("(%.1f,%.1f) ", x, y);
    point++;
}
```


for

Calculate Average Marks

Execution Example

```
Program to find the average mark of a test/quiz
```

```
How many student/marks are there: 5
```

```
Enter 5 numbers to find the averageEnter mark[1]: 80
```

```
Enter mark[2]: 72
```

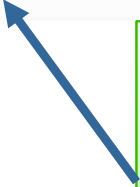
```
Enter mark[3]: 65
```

```
Enter mark[4]: 48
```

```
Enter mark[5]: 93
```

```
Sum of the 5 marks is 358
```

```
Average mark is 71
```



Note the integer result $358/5 = 71$.
Dividing two integers results in an integer,
and it truncates as opposed to rounding
the fractional part as $358 \div 5 = 71.6$

for

Calculate Average Marks

ForAverageMark.java

- The **for** based solution does the same as the **WhileAverageMark** solution.
- There are a number of changes from **WhileAverageMark**
 1. **keyboard** is now a static class variable (created once for entire class)
 2. Uses a **for** statement to enter and sum marks
 3. Find the average as a real number using a cast.
(double) casts/converts **sumMarks** from **int** to **double**
 4. Removed the unnecessary **mark** and **average** local variables
 5. Removed all comments, the code should be self explanatory.

```
Program to find the average mark of a test/quiz  
How many student/marks are there: 5
```

```
Enter 5 numbers to find the average  
Enter mark[1]: 80  
Enter mark[2]: 72  
Enter mark[3]: 65  
Enter mark[4]: 48  
Enter mark[5]: 93
```

```
Sum of the 5 marks is 358  
Average mark is 71.6
```

for

Calculate Average Marks

ForAverageMark.java

```
public class ForAverageMark
{
    private static Scanner keyboard = new Scanner( System.in );

    public static void main(String[] args)
    {
        System.out.println("Program to find the average mark of a test/quiz");

        System.out.print("How many student/marks are there: ");
        int numberMarks = keyboard.nextInt();    // read number of marks from user

        System.out.printf("\nEnter %d numbers to find the average", numberMarks);

        int sumMarks = 0;
        for(int countMarks = 1; countMarks <= numberMarks; countMarks++)
        {
            System.out.printf( "Enter mark[%d]: ", countMarks );
            sumMarks += keyboard.nextInt();
        }

        System.out.printf( "\nSum of the %d marks is %d\n", numberMarks, sumMarks );
        System.out.printf( "Average mark is %.1f\n", (double) sumMarks/numberMarks );
    }
}
```

do while

Repetition Statement

Pseudocode

*do print number
while number in range 1 to 10*

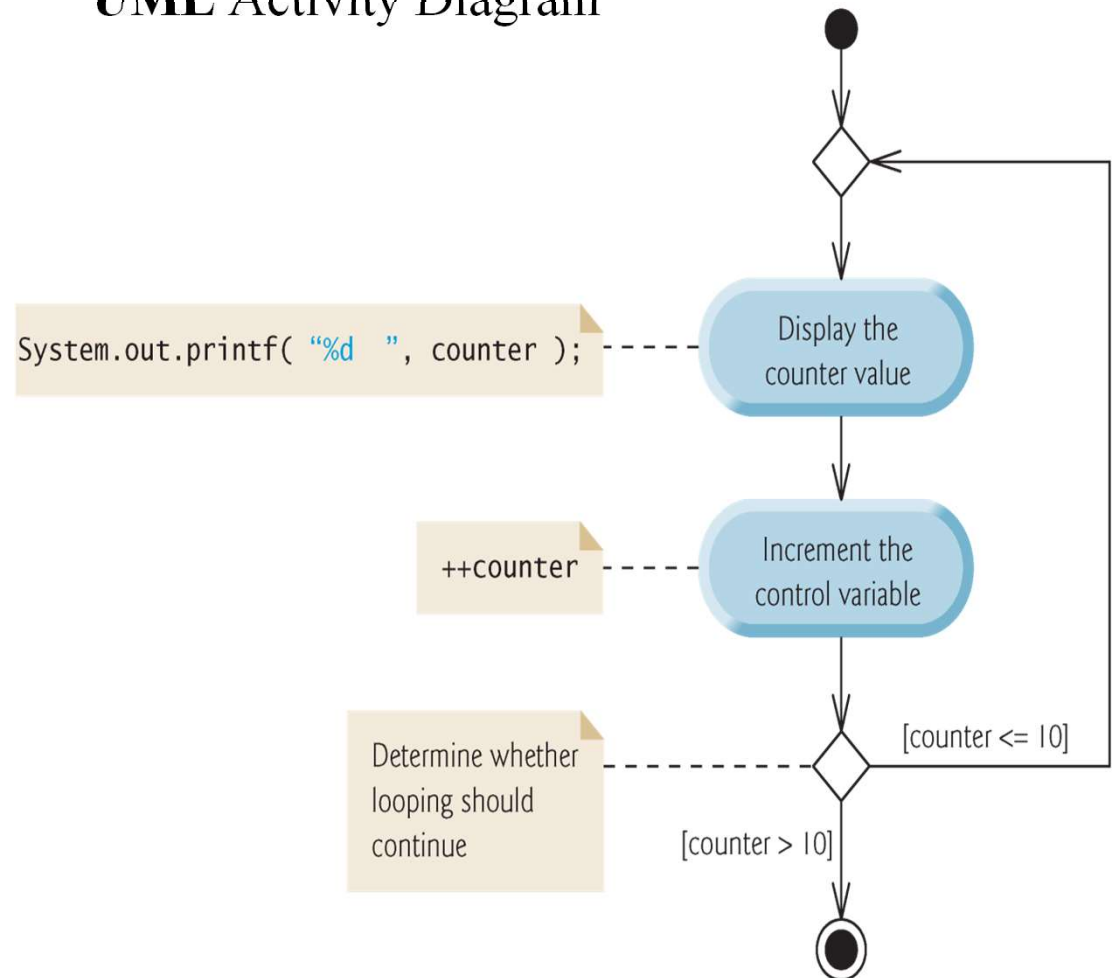
Java

```
do
{
    statement
} while (condition);
```

The **do while** iteratively executes the **statement** then checks the **condition** and if **false** the loop terminates. **Statement** is executed at least once.

```
int counter = 1;
do
{
    System.out.println( counter );
    ++counter;
} while (counter <= 10);
```

UML Activity Diagram



Output

1 2 3 4 5 6 7 8 9 10

do while

Counter-controlled repetition

DoWhileCounter.java

- **Again consider, printing the numbers from 1 to 10, one per line.**
 - Below is the solution to the same problem as **WhileCounter** and **ForCounter**.
 - It uses **counter controlled repetition**.
 - Observe that the number is first printed and then the condition (**counter <= 10**) is checked.
- Therefore, a **do while** statement executes at least once.
- Anything a **do while** can do, can also be done with a **while** or **for**
 - It may result in statement duplication.

```
public class DoWhileCounter
{
    public static void main(String[] args)
    {
        int counter = 1;
        do
        {
            System.out.println( counter );
            ++counter;
        }
        while (counter <= 10);
    }
}
```

Declare and initialise **control variable**

Loop task i.e. print counter
Increment **control variable**

Loop continuation condition

do while

More Examples

DoWhileExamples

DoWhileExamples.java

Review DoWhileExamples.java

do while code snippet examples (similar to the **while** examples):

1. Print the numbers from 1 to 10, one per line.
2. Print the numbers from 5 10 15 20 ... 50, on same line.
3. Print the numbers from 40 to 100 that are divisible by 7
4. Print the numbers from 11100 to 11200 divisible by 7
5. Sum the even numbers from 1 to 100
6. Write out the points on the line $y = 2x + 1.5$, for $x=0.0$ to 4.0 in steps of 0.5

do while

More Examples: line points

```
/*
 * 6. Write out the points on the line  $y = 2x + 1.5$ , for  $x=0.0$  to  $4.0$  in steps of  $0.5$ 
 */
{
    System.out.println( "\nPoints on a line  $y = 2x + 1.5$ , for  $x=0.0$  ..  $4.0$ , stepsize  $0.5$ ");

    int point = 0;           // Use integer variable for counting, avoid real numbers
    do
    {
        double x = point * 0.5;
        double y = 2*x + 1.5;
        System.out.printf("(%.1f,%.1f) ", x, y);
        point++;
    }
    while (point <= 8);
    System.out.println();
}
```

- Compare with **while** equivalent in WhileExamples.java

```
int point = 0;
while (point <= 8)
{
    double x = point * 0.5;
    double y = 2*x + 1.5;
    System.out.printf("(%.1f,%.1f) ", x, y);
    point++;
}
```


do while

Get a number in range

DoWhileGetNumber.java

- Define a method to get a number in the range e.g. 1 to 100.
- A natural **do while** as we must do something at least once i.e. get a number

```
public class DoWhileGetNumber
{
    public static void main(String[] args)
    {
        System.out.println( "Here's a number in the range [1,100] " + getNumber(1, 100) );
    }

    public static int getNumber(int lower, int upper)
    {
        Scanner keyboard = new Scanner(System.in);
        int number;
        do
        {
            System.out.printf("Enter a number in the range %d to %d inclusive: ", lower, upper);
            number = keyboard.nextInt();
            // Here would inform the user that the number was out of range, and invite retry
        } while (number < lower || number > upper);
        return number;
    }
}
```


break and continue statements

break statement

- Must be used in a **while**, **for**, **do while** or **switch control statements**.
- When executed it causes an immediate exit from that statement.
- The execution will continue with the first statement after the control statement.
- The uses of the **break** statement are:
 - To terminate a loop early (escape by overriding from the loop condition).
 - To skip the remainder of a **switch** statement (more later...)

continue statement

- Must be used in a repetition **while**, **for** or **do while control statements**.
- When executed in it skips the remaining statements in the loop body.
- It proceeds with the next iteration of the loop.
- In **while** and **do...while** statements, the **condition** expression is next executed.
- In a **for** statement, the **increment** expression is next executed, and then the **condition** is evaluated as normal.

The uses of the **break** and **continue** statements are limited, avoidable and discouraged.

break and continue statements

Example

BreakContinue.java

```
public class BreakContinue
{
    public static void main(String[] args)
    {
        int counter;

        for (counter = 1; counter <= 6; counter++)
        {
            if (counter==4)
                break;           // terminate the loop
            System.out.println( counter );
        }
        System.out.printf("for break terminated with counter=%d\n\n", counter);

        for (counter = 1; counter <= 6; counter++)
        {
            if (counter==4)
                continue;        // skip rest of loop body
            System.out.println( counter );
        }
        System.out.printf("for continue completed with counter=%d\n", counter);
    }
}
```

```
1
2
3
for break terminated with counter=4
4
5
6
for continue completed with counter=7
```

switch

Selection statement

- The **switch** statement is a **multiple-selection statement** which performs different actions based on the switch **expression** and the values of **case** labels.
- The **switch** keyword is followed by an **expression** in () and a statement block { } that contains a sequence of **case** labels and an optional **default case**.
- The labels can be **constant expressions** of type **byte**, **short**, **int**, **long**, **char** or **String**
- A **switch** statement can always be replaced with an **if/else/if** ladder

```
switch ( expression )
{
    case label_1:      statement(s)_for_label_1
                       break;
    case label_2:      statement(s)_for_label_2
                       break;

    default:           statement(s)_for_default // Optional
                       break;
}
```

A **switch** can use certain other reference types such as **Integer**, **Character** etc. and **enumerated types** (more later!).

switch

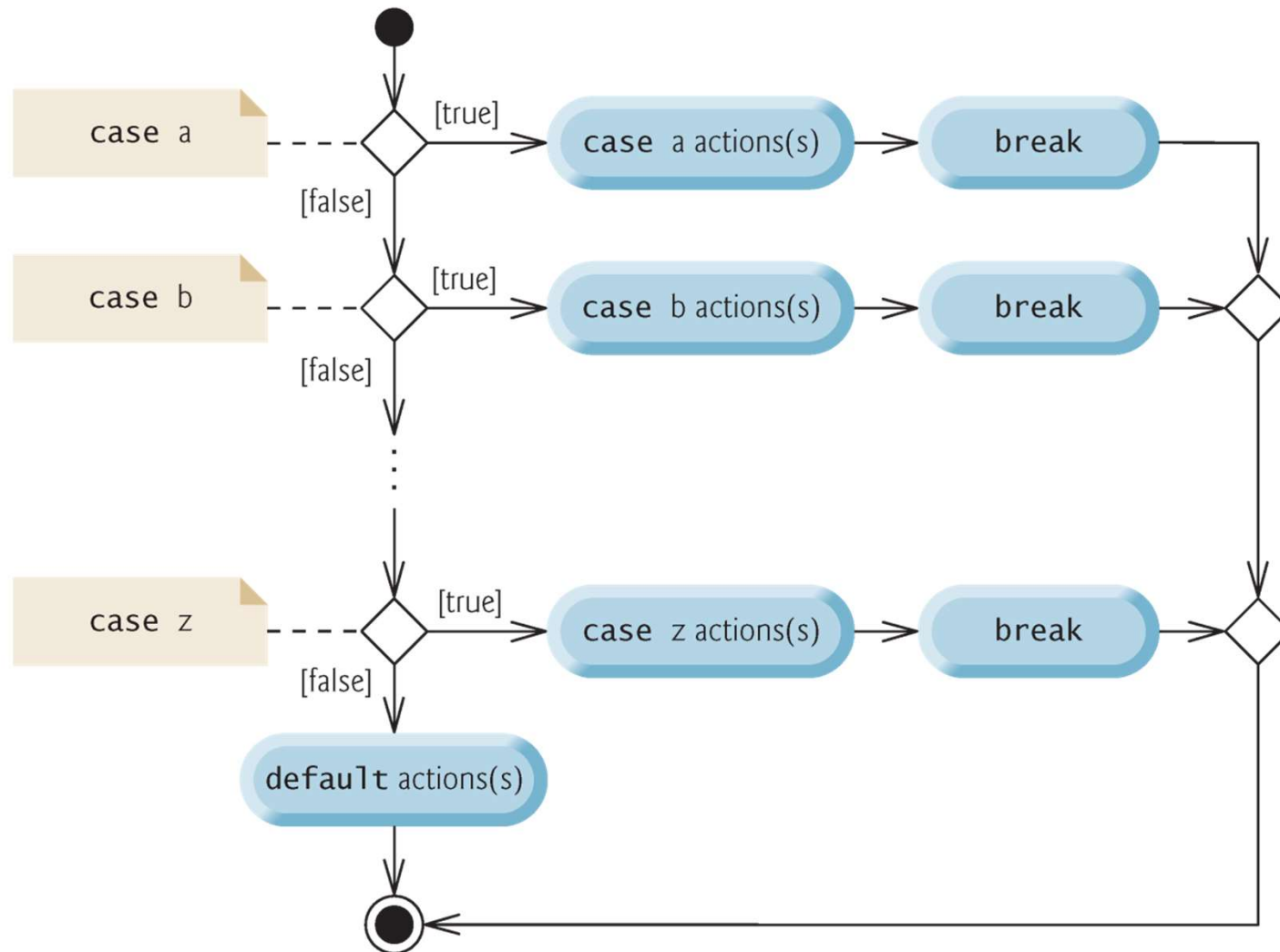
Selection statement

- The program evaluates the controlling **switch expression** in the parentheses following keyword **switch**.
- The program compares the expression's value with each **case** label.
- If a match occurs, the program executes that **case**'s statements until it meets a **break statement** or reaches the end of the switch block **}**.
- The **break statement** causes program control to proceed with the first statement after the switch.
- Each **case** can have multiple statements.
- You can use **case** constants that are integral, character 'A', or String "Hola".
- You can use **constant expressions** in a **case** label, as long as it yields a integral primitive type or a String, and whose value can be evaluated at compile time to a literal e.g $1+8/4$ (3), "Hi" + "Ho" ("HiHo").

Omitting a **break** statement, may result in a **logic error** where a matched **case** continues processing the statements of a subsequent **case**.

switch statement

Example UML activity diagram



switch statement

Simple menu

SwitchMenu

SwitchMenu.java

```
System.out.println("A very simple menu");
System.out.println("1. The 1st menu item");
System.out.println("2. The 2nd menu item");
System.out.println("3. The 3rd menu item");
System.out.println();
```

```
Scanner keyboard = new Scanner(System.in);
System.out.print("Choice: ");
```

```
switch ( keyboard.nextInt() )
{
    case 1:
        System.out.println("How boring, you chose the 1st menu option");
        break;
    case 2:
        System.out.println("How exciting, you chose the 2nd menu option");
        break;
    case 3:
        System.out.println("Unbelievable, you chose the 3rd menu option");
        break;
    default:
        System.out.println("Not a valid menu choice");
        break;
}
```

Display a menu

Input choice

Print String based on choice

switch statement

Math class methods and Random Numbers

SwitchMath.java 1 of 3

- Separate methods to display the menu and get the item selected

```
/**
 * Display the menu items as a list
 */
public static void displayMenuItems()
{
    System.out.println();
    System.out.println("1. Explore Java Math methods for  $\sqrt{\phantom{x}}$ , log10, max, min");
    System.out.println("2. Explore Java Math trigonometry methods");
    System.out.println("3. Explore Java Math exponentiation, log and power methods");
    System.out.println("4. Display a random dice face number");
    System.out.println("5. Display a random 24 hour time");
    System.out.println("6. Exit");
    System.out.println();
}
```

```
/**
 * Get an menu item choice from the menu
 * @return menu item choice
 */
public static int getMenuitem()
{
    Scanner keyboard = new Scanner(System.in);
    System.out.print("Choice: ");
    return keyboard.nextInt();
}
```


switch statement

SwitchMath

Math class methods and Random Numbers

SwitchMath.java 2 of 3

```
public static void main(String[] args)
{
    boolean isContinue = true;

    System.out.println("A simple menu");
    while (isContinue)
    {
        displayMenuItems();

        switch ( getMenuItem() )
        {
            case 1:
                System.out.println("java Math class include sqrt, log10, min, max");
                System.out.printf(" √1.44          \t⇒ Math.sqrt(1.44)      = %.2f\n", Math.sqrt(1.44) );
                System.out.printf(" log101000      \t⇒ Math.log10(1000)      = %.2f\n", Math.log10(1000.0) );
                System.out.printf(" max(4,8)       \t⇒ Math.max(4,8)       = %d\n", Math.max(4,8));
                System.out.printf(" min(3.2,-1.5) \t⇒ Math.min(3.2, -1.5) = %.2f\n", Math.min(3.2, -1.5));
                break;

            case 2:
                // Note angles in Java are in radians 45° = π/4
                System.out.println("Java π as represented by Math.PI constant is " + Math.PI );
                System.out.printf(" cos(45°) = cos(π/4) ⇒ Math.cos(Math.PI/4) = %.3f\n", Math.cos(Math.PI/4.0) );
                System.out.printf(" sin(30°) = sin(π/6) ⇒ Math.cos(Math.PI/6) = %.3f\n", Math.sin(Math.PI/6.0) );
                System.out.printf(" tan(60°) = tan(π/3) ⇒ Math.tan(Math.PI/3) = %.3f\n", Math.tan(Math.PI/3.0) );
                break;
        }
    }
}
```

switch statement

SwitchMath

Math class methods and Random Numbers

SwitchMath.java 3 of 3

```
case 3:
    System.out.println("Java Euler's constant e as presented by Math.E constant is " + Math.E );
    System.out.printf(" e^4 \t⇒ Math.exp(4.0)           = %.3f\n", Math.exp(4.0) );
    System.out.printf(" e^4 \t⇒ Math.pow(Math.E, 4.0) = %.3f\n", Math.pow(Math.E, 4.0) );
    System.out.printf(" log_e \t⇒ Math.log(Math.E)      = %.3f\n", Math.log(Math.E));
    break;

case 4:
    Random rng1 = new Random();          // A random number generator for a dice
    System.out.printf("A random dice face is: %2d\n", 1 + rng1.nextInt(6) );
    break;

case 5:
    Random rng2 = new Random();          // for a random 24h time, could also use rng1 here
    System.out.printf("A random 24h time is: %02d:%02d\n", rng2.nextInt(24), rng2.nextInt(60) );
    break;

case 6:
    System.out.println("Bye, stay safe out there");
    isContinue = false;
    break;

default:
    System.out.println("*** Not a valid choice, try again");
    break;
}
```

switch statement

Explore some Date/Time classes

SwitchDateTime

SwitchDateTime.java

```
import java.time.LocalDateTime;    // Use LocalDateTime, LocalDate and LocalTime
import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;

import java.util.Scanner;
```

```
public class SwitchDateTimes
{
    /**
     * Display menu and process choice until user selects exit
     * @params args command line arguments (not used)
     */
    public static void main(String[] args)
    {
        System.out.println("Explore date and time menu");
        do
        {
            displayMenuItems();
        }
        while ( doMenuChoice() );
    }

    /**
     * Get a new menu choice and process it
     * @return false if exit selected, true otherwise
     */
    public static boolean doMenuChoice()
    {
        boolean isContinue = true;
```

The **SwitchDateTime** class gives further insight into creating a switch menu and explores some functionality of the extensive Java classes to support dates and times. For more details, refer to:
SwitchDateTime.java

Explore date and time menu

- A. Display the current date and time (default)
- B. Display the current date and time (formatted)
- C. Display the current date (default/formatted)
- D. Display the current time (default/formatted)
- E. Exit

Choice: c

Today's date is (default format): 2020-11-08
(dd-MMM-yyyy formatter): 08-Nov-2020